

Instituto Politécnico Nacional
Escuela Superior de Cómputo
Ingeniería en
Sistemas Computacionales

Unidad de Aprendizaje:
Inteligencia Artificial
Grupo: 6CV2

Práctica: Menú de algoritmos
de búsqueda

Alumno:
Barros Martínez Luis Enrique
Bautista Ríos Alfredo
Garcia Martinez Angel Yahel

Profesor: Marco Antonio Castillo

Introducción

El programa presentado es una implementación visual e interactiva de varios algoritmos de búsqueda en grafos, utilizando **SDL2** como biblioteca gráfica. Permite a los usuarios experimentar con diferentes estrategias de búsqueda, como BFS, DFS, Dijkstra y A*, en un laberinto representado como una cuadrícula. Los usuarios pueden seleccionar un punto de inicio y un objetivo, y el programa calcula y visualiza el camino más corto entre ellos según el algoritmo elegido.

El diseño combina funcionalidad y facilidad de uso, proporcionando botones para elegir el algoritmo deseado y un botón de reinicio para realizar múltiples experimentos. Además, el programa muestra visualmente el progreso y el resultado de cada algoritmo, destacando su comportamiento y diferencias en tiempo de ejecución y eficiencia en la búsqueda de rutas.

Desarrollo

Estructura del Programa

El programa se divide en varias secciones principales, cada una enfocada en un aspecto específico:

1. Interfaz Gráfica:

La interfaz gráfica se implementa utilizando **SDL2** para renderizar el laberinto y los elementos interactivos. Algunos elementos clave son:

- **Botones:** Se encuentran en una columna lateral y permiten seleccionar los algoritmos BFS, DFS, A*, y Dijkstra, además de un botón de reinicio.
- **Laberinto:** Representado como una cuadrícula donde las celdas son obstáculos (#) o espacios libres (_). Las celdas se colorean según su rol (inicio, objetivo, camino, o espacio libre).

2. Lógica de Búsqueda:

La clase **Pathfinding** encapsula la implementación de los algoritmos:

- **BFS (Breadth-First Search):** Explora los nodos por nivel, garantizando encontrar el camino más corto en grafos no ponderados.
- **DFS (Depth-First Search):** Explora nodos profundamente antes de retroceder, siendo menos eficiente para encontrar caminos cortos pero útil en ciertas aplicaciones.

- **A (A Star):*** Combina la exploración por costo acumulado y una heurística (distancia Manhattan) para encontrar caminos óptimos de manera eficiente.
- **Dijkstra:** Variante que utiliza la distancia mínima acumulada, idéntica a A* cuando se omite la heurística.

3. Interactividad del Usuario:

Los usuarios pueden seleccionar un punto de inicio y un objetivo haciendo clic en el laberinto. Los botones permiten alternar entre algoritmos y reiniciar la simulación.

Detalles de Implementación

1. Representación del Laberinto:

El laberinto es una matriz bidimensional de caracteres (**maze**), donde cada celda indica si es un obstáculo (**#**) o un espacio libre (**_**). Esto simplifica la validación de movimientos en los algoritmos.

2. Algoritmos de Búsqueda:

- **BFS y DFS:** Implementados utilizando estructuras estándar (**std::queue** y **std::stack**) para explorar nodos en diferentes órdenes. Ambos reconstruyen el camino al objetivo utilizando una estructura **cameFrom** que rastrea el nodo anterior de cada celda visitada.
- **A y Dijkstra:*** Usan una cola de prioridad para explorar nodos según el costo estimado más bajo. A* combina el costo acumulado con una heurística, mientras que Dijkstra utiliza solo el costo acumulado.
- **Reutilización de Código:** La implementación de Dijkstra reutiliza A* al ignorar la heurística.

3. Visualización del Camino:

Durante la ejecución de un algoritmo, el camino se dibuja progresivamente con un retardo (**SDL_Delay**) para ilustrar el proceso. Las celdas en el camino se colorean de azul, mientras que el inicio y el objetivo permanecen destacados en verde y rojo, respectivamente.

4. Interfaz Gráfica:

La función **drawMaze** es responsable de renderizar todo el laberinto, los botones y los caminos generados. También gestiona los colores según el estado de cada celda.

Flujo del Programa

1. El programa comienza con la inicialización de SDL2 y la configuración de la ventana.
2. Los usuarios seleccionan el inicio y el objetivo en el laberinto, y luego eligen un algoritmo mediante los botones.
3. El algoritmo seleccionado calcula el camino más corto entre los puntos y lo dibuja progresivamente.
4. Los usuarios pueden reiniciar la simulación para probar con diferentes configuraciones.

Conclusión

Este programa combina la visualización interactiva con la funcionalidad de varios algoritmos de búsqueda en grafos, proporcionando una herramienta educativa y experimental. Su estructura modular facilita la adición de nuevos algoritmos o ajustes en el laberinto. La visualización progresiva del camino ayuda a entender el comportamiento de cada algoritmo, destacando sus diferencias en eficiencia y exploración.

El uso de SDL2 para la interfaz gráfica asegura una representación clara y atractiva del laberinto y los caminos generados. En conjunto, este programa es una excelente plataforma para aprender y explorar técnicas de búsqueda en grafos, con aplicaciones prácticas en navegación, planificación de rutas y simulaciones.

Código

```
#include <SDL2/SDL.h>
#include <SDL2/SDL_ttf.h>
#include <iostream>
#include <algorithm>
#include <queue>
#include <stack>
#include <vector>
#include <cmath>
#include <map>

const int WINDOW_WIDTH = 800;
```

```
const int WINDOW_HEIGHT = 600;
const int BUTTON_WIDTH = 200;
const int GRID_SIZE = 30;
const int CELL_SIZE = 20;

// Colores
const SDL_Color WHITE = {255, 255, 255, 255};
const SDL_Color BLACK = {0, 0, 0, 255};
const SDL_Color GREEN = {0, 255, 0, 255};
const SDL_Color RED = {255, 0, 0, 255};
const SDL_Color BLUE = {0, 0, 255, 255};
const SDL_Color GRAY = {200, 200, 200, 255};

// Estados
enum State { SELECT_START, SELECT_GOAL, RUNNING };

// Laberinto
char maze[GRID_SIZE][GRID_SIZE] = {
```

```
};

// Función para dibujar texto (requiere SDL_ttf)
void drawText(SDL_Renderer* renderer, const char* text, int x, int y, SDL_Color color, int fontSize) {
    TTF_Font* font = TTF_OpenFont("../SDL_Visor/fonts/monospace/Monospace.ttf",
fontSize); // Asegúrate de que la ruta al archivo de la fuente sea correcta
    if (font == nullptr) {
        std::cerr << "Failed to load font: " << TTF_GetError() << std::endl;
        return;
    }
    SDL_Surface* surface = TTF_RenderText_Solid(font, text, color);
    if (surface == nullptr) {
        std::cerr << "Failed to create text surface: " << TTF_GetError() <<
std::endl;
        TTF_CloseFont(font);
        return;
    }
    SDL_Texture* texture = SDL_CreateTextureFromSurface(renderer, surface);
    SDL_FreeSurface(surface);
    if (texture == nullptr) {
        std::cerr << "Failed to create text texture: " << SDL_GetError() <<
std::endl;
        TTF_CloseFont(font);
    }
}
```

```

        return;
    }

    SDL_Rect destRect = {x, y, surface->w, surface->h};
    SDL_RenderCopy(renderer, texture, nullptr, &destRect);
    SDL_DestroyTexture(texture);
    TTF_CloseFont(font);
}

// Clase para manejar los algoritmos de búsqueda
class Pathfinding {
public:
    static std::vector<std::pair<int, int>> BFS(const std::pair<int, int>& start,
const std::pair<int, int>& goal);

    static std::vector<std::pair<int, int>> DFS(const std::pair<int, int>& start,
const std::pair<int, int>& goal);

    static std::vector<std::pair<int, int>> Dijkstra(const std::pair<int, int>&
start, const std::pair<int, int>& goal);

    static std::vector<std::pair<int, int>> AStar(const std::pair<int, int>& start,
const std::pair<int, int>& goal);
};

// Verifica si una celda es válida
bool isValidCell(int x, int y);

// Dibuja el laberinto
void drawMaze(SDL_Renderer* renderer, const std::pair<int, int>& start, const
std::pair<int, int>& goal, const std::vector<std::pair<int, int>>& path);

// Programa principal
int main(int argc, char* argv[]) {
    SDL_Init(SDL_INIT_VIDEO);

    TTF_Init();

    SDL_Window* window = SDL_CreateWindow("Pathfinding Visualization",
SDL_WINDOWPOS_CENTERED, SDL_WINDOWPOS_CENTERED, WINDOW_WIDTH, WINDOW_HEIGHT,
SDL_WINDOW_SHOWN);

    SDL_Renderer* renderer = SDL_CreateRenderer(window, -1,
SDL_RENDERER_ACCELERATED);

    bool running = true;
    State state = SELECT_START;

    std::pair<int, int> start = {-1, -1};
    std::pair<int, int> goal = {-1, -1};
    std::vector<std::pair<int, int>> path;

    char selectedAlgorithm = 0;
    while (running) {
        SDL_Event event;

        while (SDL_PollEvent(&event)) {
            if (event.type == SDL_QUIT) {
                running = false;
            } else if (event.type == SDL_MOUSEBUTTONDOWN) {
                int x = event.button.x;

```

```

        int y = event.button.y;
        if (x < BUTTON_WIDTH) { // Botones
            if (y < 100) selectedAlgorithm = 'B'; // BFS
            else if (y < 200) selectedAlgorithm = 'D'; // DFS
            else if (y < 300) selectedAlgorithm = 'A'; // A*
            else if (y < 400) selectedAlgorithm = 'J'; // Dijkstra
            else if (y >= 400 && y < 500) { // Botón de reset
                start = {-1, -1};
                goal = {-1, -1};
                path.clear();
                state = SELECT_START;
            }

            if (selectedAlgorithm && start.first != -1 && goal.first != -1)
            {
                switch (selectedAlgorithm) {
                    case 'B': path = Pathfinding::BFS(start, goal); break;
                    case 'D': path = Pathfinding::DFS(start, goal); break;
                    case 'A': path = Pathfinding::AStar(start, goal);
break;

                    case 'J': path = Pathfinding::Dijkstra(start, goal);
break;

                }

                state = RUNNING;
            }
        } else { // Laberinto
            int gridX = (x - BUTTON_WIDTH) / CELL_SIZE;
            int gridY = y / CELL_SIZE;
            if (state == SELECT_START) {
                start = {gridX, gridY};
                state = SELECT_GOAL;
            } else if (state == SELECT_GOAL) {
                goal = {gridX, gridY};
            }
        }
    }

    SDL_SetRenderDrawColor(renderer, WHITE.r, WHITE.g, WHITE.b, WHITE.a);
    SDL_RenderClear(renderer);

    // Dibujar botones
    SDL_SetRenderDrawColor(renderer, GRAY.r, GRAY.g, GRAY.b, GRAY.a);
    SDL_Rect buttonRect = {0, 0, BUTTON_WIDTH, 100};
    for (int i = 0; i < 4; ++i) {
        SDL_RenderFillRect(renderer, &buttonRect);
    }
}

```

```

        buttonRect.y += 100;
    }
    // Dibujar el botón de reset
    SDL_Rect resetButtonRect = {0, 400, BUTTON_WIDTH, 100};
    SDL_RenderFillRect(renderer, &resetButtonRect);
    drawText(renderer, "Reset", 50, 430, BLACK, 24);
    drawText(renderer, "BFS", 50, 30, BLACK, 24);
    drawText(renderer, "DFS", 50, 130, BLACK, 24);
    drawText(renderer, "A*", 50, 230, BLACK, 24);
    drawText(renderer, "Dijkstra", 50, 330, BLACK, 24);
    // Dibujar laberinto
    drawMaze(renderer, start, goal, path);
    SDL_RenderPresent(renderer);
}

TTF_Quit();
SDL_DestroyRenderer(renderer);
SDL_DestroyWindow(window);
SDL_Quit();
return 0;
}

bool isValidCell(int x, int y) {
    return x >= 0 && x < GRID_SIZE && y >= 0 && y < GRID_SIZE && maze[y][x] == '_';
}

void drawMaze(SDL_Renderer* renderer, const std::pair<int, int>& start, const
std::pair<int, int>& goal, const std::vector<std::pair<int, int>>& path) {
    for (int y = 0; y < GRID_SIZE; ++y) {
        for (int x = 0; x < GRID_SIZE; ++x) {
            SDL_Rect cell = {BUTTON_WIDTH + x * CELL_SIZE, y * CELL_SIZE,
CELL_SIZE, CELL_SIZE};
            SDL_SetRenderDrawColor(renderer, WHITE.r, WHITE.g, WHITE.b, WHITE.a);
            if (maze[y][x] == '#') SDL_SetRenderDrawColor(renderer, BLACK.r,
BLACK.g, BLACK.b, BLACK.a);
            SDL_RenderFillRect(renderer, &cell);
            SDL_SetRenderDrawColor(renderer, GRAY.r, GRAY.g, GRAY.b, GRAY.a);
            SDL_RenderDrawRect(renderer, &cell);
        }
    }

    // Dibujar inicio y objetivo
    if (start.first != -1) {
        SDL_Rect cell = {BUTTON_WIDTH + start.first * CELL_SIZE, start.second *
CELL_SIZE, CELL_SIZE, CELL_SIZE};
        SDL_SetRenderDrawColor(renderer, GREEN.r, GREEN.g, GREEN.b, GREEN.a);
    }
}

```

```

        SDL_RenderFillRect(rendererer, &cell);
    }

    if (goal.first != -1) {
        SDL_Rect cell = {BUTTON_WIDTH + goal.first * CELL_SIZE, goal.second *
CELL_SIZE, CELL_SIZE, CELL_SIZE};

        SDL_SetRenderDrawColor(rendererer, RED.r, RED.g, RED.b, GREEN.a);
        SDL_RenderFillRect(rendererer, &cell);
    }

    // Dibujar camino con un delay sin parpadeos
    for (size_t i = 0; i < path.size(); ++i) {
        // Limpiar la pantalla
        //SDL_SetRenderDrawColor(rendererer, 0, 0, 0, 255); // Fondo negro
        //SDL_RenderClear(rendererer);

        // Redibujar el entorno completo
        // Redibujar el inicio
        if (start.first != -1) {
            SDL_Rect startCell = {BUTTON_WIDTH + start.first * CELL_SIZE,
start.second * CELL_SIZE, CELL_SIZE, CELL_SIZE};

            SDL_SetRenderDrawColor(rendererer, GREEN.r, GREEN.g, GREEN.b, GREEN.a);
            SDL_RenderFillRect(rendererer, &startCell);
        }

        // Redibujar el objetivo
        if (goal.first != -1) {
            SDL_Rect goalCell = {BUTTON_WIDTH + goal.first * CELL_SIZE, goal.second
* CELL_SIZE, CELL_SIZE, CELL_SIZE};

            SDL_SetRenderDrawColor(rendererer, RED.r, RED.g, RED.b, RED.a);
            SDL_RenderFillRect(rendererer, &goalCell);
        }

        // Dibujar el camino actual hasta el punto i
        SDL_SetRenderDrawColor(rendererer, BLUE.r, BLUE.g, BLUE.b, BLUE.a);
        for (size_t j = 0; j <= i; ++j) {
            SDL_Rect cell = {BUTTON_WIDTH + path[j].first * CELL_SIZE,
path[j].second * CELL_SIZE, CELL_SIZE, CELL_SIZE};

            SDL_RenderFillRect(rendererer, &cell);
        }

        // Mostrar la pantalla actualizada
        SDL_RenderPresent(rendererer);

        // Añadir el delay
    }

```

```

        SDL_Delay(200);
    }

}

std::vector<std::pair<int, int>> Pathfinding::BFS(const std::pair<int, int>& start,
const std::pair<int, int>& goal) {
    std::queue<std::pair<int, int>> q;
    std::map<std::pair<int, int>, std::pair<int, int>> cameFrom;
    q.push(start);
    cameFrom[start] = {-1, -1};
    const std::vector<std::pair<int, int>> directions = {{0, 1}, {1, 0}, {0, -1},
{-1, 0}};

    while (!q.empty()) {
        auto current = q.front();
        q.pop();
        if (current == goal) {
            // Reconstruir el camino
            std::vector<std::pair<int, int>> path;
            for (auto at = goal; at != std::pair<int, int>{-1, -1}; at =
cameFrom[at]) {
                path.push_back(at);
            }
            std::reverse(path.begin(), path.end());
            return path;
        }
        for (const auto& dir : directions) {
            std::pair<int, int> neighbor = {current.first + dir.first,
current.second + dir.second};
            if (isValidCell(neighbor.first, neighbor.second) &&
cameFrom.find(neighbor) == cameFrom.end()) {
                q.push(neighbor);
                cameFrom[neighbor] = current;
            }
        }
    }

    return {}; // No se encontró un camino
}

std::vector<std::pair<int, int>> Pathfinding::DFS(const std::pair<int, int>& start,
const std::pair<int, int>& goal) {
    std::stack<std::pair<int, int>> s;
    std::map<std::pair<int, int>, std::pair<int, int>> cameFrom;
    s.push(start);
    cameFrom[start] = {-1, -1};

```

```

    const std::vector<std::pair<int, int>> directions = {{0, 1}, {1, 0}, {0, -1},
{-1, 0}};

    while (!s.empty()) {
        auto current = s.top();
        s.pop();
        if (current == goal) {
            std::vector<std::pair<int, int>> path;
            for (auto at = goal; at != std::pair<int, int>{-1, -1}; at =
cameFrom[at]) {
                path.push_back(at);
            }
            std::reverse(path.begin(), path.end());
            return path;
        }

        for (const auto& dir : directions) {
            std::pair<int, int> neighbor = {current.first
+ dir.first, current.second + dir.second};

            if (isValidCell(neighbor.first,
neighbor.second) && cameFrom.find(neighbor) ==
cameFrom.end()) {
                s.push(neighbor);
                cameFrom[neighbor] = current;
            }
        }
    }

    return {};
}

std::vector<std::pair<int, int>> Pathfinding::AStar(const
std::pair<int, int>& start, const std::pair<int, int>&
goal) {
    auto heuristic = [](const std::pair<int, int>& a,
const std::pair<int, int>& b) {
        return abs(a.first - b.first) + abs(a.second -
b.second);
    };

```

```

};

std::priority_queue<std::pair<int, std::pair<int,
int>>>,

std::vector<std::pair<int,
std::pair<int, int>>>>,

std::greater<>> pq;

std::map<std::pair<int, int>, int> gScore;

std::map<std::pair<int, int>, std::pair<int, int>>
cameFrom;

gScore[start] = 0;

pq.push({heuristic(start, goal), start});

cameFrom[start] = {-1, -1};

const std::vector<std::pair<int, int>> directions =
{{0, 1}, {1, 0}, {0, -1}, {-1, 0}};

while (!pq.empty()) {
    auto current = pq.top().second;
    pq.pop();

    if (current == goal) {
        std::vector<std::pair<int, int>> path;

        for (auto at = goal; at != std::pair<int,
int>{-1, -1}; at = cameFrom[at]) {
            path.push_back(at);
        }

        std::reverse(path.begin(), path.end());
        return path;
    }

    for (const auto& dir : directions) {
        std::pair<int, int> neighbor = {current.first
+ dir.first, current.second + dir.second};
    }
}

```

```

        if (isValidCell(neighbor.first,
neighbor.second)) {
            int tentativeGScore = gScore[current] + 1;
            if (gScore.find(neighbor) == gScore.end()
|| tentativeGScore < gScore[neighbor]) {
                gScore[neighbor] = tentativeGScore;
                pq.push({tentativeGScore +
heuristic(neighbor, goal), neighbor});
                cameFrom[neighbor] = current;
            }
        }
    }
}

return {};
}

std::vector<std::pair<int, int>>
Pathfinding::Dijkstra(const std::pair<int, int>& start,
const std::pair<int, int>& goal) {
    return AStar(start, goal);
}

```

Implementacion Comparativo:



Implementación Menu



